# Watching Models Learn

Eliminating the Round Trip Between Training and Seeing

A whitepaper on the Caliper platform and its zero-copy tensor path

Ahmed Khan

July 2026 — draft v0.3 (LaTeX edition)

**One-sentence thesis.** Every mainstream tool for looking inside a neural network while it trains copies the data off the GPU, through the CPU, usually onto disk, and back onto a GPU inside a browser — a round trip measured in seconds; Caliper deletes that round trip, so the model's internal state is drawn on screen *in the same frame it was computed*, and this is verified pixel-exact on both of the dominant hardware ecosystems (Apple Silicon and Windows/NVIDIA).

## How to read this paper

| You are | Read | You will come away with |
|---|---|---|
| **General reader / non-technical** | §1, §2, §4 | Why "watching a model learn, live" was impossible yesterday and what it looks like today |
| **Business / product / investment** | §1, §2, §4, §5 | What the round trip costs in GPU-hours, engineer-hours, and infrastructure — and why deleting it is defensible |
| **Research / engineering** | §1, §3, §6–§9, appendices | The mechanism (memory aliasing, external-memory interop, the sync ladder), the verification method, and the honest limits |

Every section is self-contained; no section assumes you read the deeper ones.

## 1 Executive summary

Training a neural network is the most instrumented-*around* and least instrumented-*inside* process in modern computing. We log the loss, the learning rate, the GPU temperature — everything **about** the run — while the thing actually being made, the model's internal state, sits on the GPU in a form nobody looks at until the run is over, or at best minutes behind.

The reason is mechanical, not conceptual. The internal state lives in GPU memory. Every mainstream visualization tool (TensorBoard and its descendants) gets it to your eyes by: copying it to CPU memory, encoding it to an image file, writing that to disk, polling the file from a web server, decoding it in a browser, and uploading it *back onto a GPU* to display. Two bus crossings, one filesystem, one encode/decode, seconds of latency — for data that started out already on the chip that renders your screen.

**Caliper removes the round trip.** The tensor's bytes never leave the GPU and are never staged through CPU memory. On Apple Silicon, the training framework's tensor and the renderer's texture are *the same physical memory* (unified memory makes this a pointer cast, plus safety rules that make the cast honest). On Windows/NVIDIA, compute and graphics APIs are made to share one VRAM allocation via external-memory interop, synchronized by GPU-side timeline semaphores — no CPU thread waits on the hot path. In both cases the result is on

screen **the same frame it was computed**, which is what makes watching weights reorganize at 60 fps — every optimizer step, not every checkpoint — possible at all.

Three claims, one per audience:

- **For the curious reader:** this turns training from developing photographs into watching a live video feed — you *see* a network sort a cloud of handwritten digits into ten colored lobes as it learns, in real time.
- **For the business reader:** the round trip is paid in wasted GPU-hours (failures discovered late), engineer time (log-out, load-in-a-notebook archaeology), and infrastructure (a logging server, a browser stack, disk churn). Caliper is a native desktop application with none of that footprint, and it runs on both hardware ecosystems that matter.
- **For the researcher:** in-the-loop, per-step observability is a new instrument class for interpretability — logit lenses, attention-head specialization trajectories, and embedding-space dynamics rendered live during training rather than reconstructed afterward. Every claim in this paper is backed by a pixel-exact automated test against a CPU reference, on real hardware, on both platforms.

## 2 The problem, in plain terms

*(Primary audience: everyone. This section deliberately contains no API names.)*

A neural network learns by adjusting millions of internal numbers, thousands of times per minute. Those numbers have structure — early vision filters literally *look like* edge detectors; a language model's attention heads develop visible specializations. Watching that structure emerge is not a luxury; it is often the fastest way to know whether a run is healthy, and for a researcher it is the phenomenon under study itself.

But here is what "watching" actually involves today, against what this paper proposes (Figure 1):

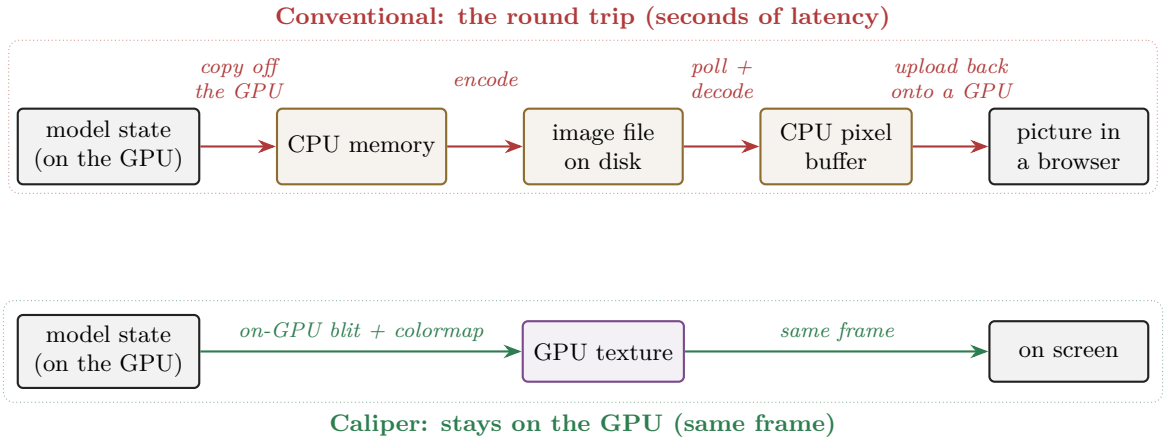


**Figure 1:** The round trip every mainstream tool pays, against Caliper's same-frame path.

It is as if a security camera worked by printing each frame, mailing the photograph, and scanning it at the other end. The picture arrives; the *event* is long over. In practice this means researchers sample sparsely (every $N$ minutes, every epoch), and whole categories of fast dynamics — what happens in the first two hundred optimizer steps, how a representation reorganizes during a loss spike — are simply never seen by anyone.

The strange part: the data starts out on the same class of chip that draws your screen. The round trip exists not because physics demands it, but because the software worlds of *computing*

on a GPU and *drawing* with a GPU grew up separately and rarely share memory. That accident of history is the entire problem Caliper solves.

**What "solved" looks like:** you click *Train* and watch a three-dimensional cloud of 2,000 handwritten digits — positioned by the network's own internal representation — split from one gray blob into ten colored lobes as the model learns to tell digits apart. Rotatable with a mouse, updating live, on a laptop. No server. No browser. No files.

## 3 Why nobody fixed this before

*(Primary audience: technical readers; business readers get the moat argument in §5. This section is what separates an external paper from internal docs — it must establish that the problem is genuinely hard, not merely neglected.)*

Three walls stand between a training loop and a rendered pixel, and each one independently forces the CPU round trip in existing tools:

1. **The API wall.** Compute frameworks speak CUDA/MPS; renderers speak Vulkan/Metal/OpenGL. These APIs historically could not reference each other's memory. The escape hatches — unified memory on Apple Silicon, `VK_KHR_external_memory` + CUDA interop on NVIDIA — are recent, fiddly, platform-specific, and rarely used outside game engines.

2. **The process wall.** The standard tooling architecture puts visualization in a *different process* (a web server) and often a different machine. Once you cross a process boundary, you are serializing; once you serialize GPU-resident data, the round trip is already lost.

3. **The correctness wall.** Sharing memory between a training framework that is *still writing* and a renderer that is *reading* is a race by default. Getting it wrong doesn't crash politely; it renders yesterday's bytes or corrupts command streams. (We hit both — §7 documents the two races and the rules that survived them.)

Each wall has a known workaround; the contribution is an architecture in which all three are crossed *at once*, portably, behind a contract small enough to freeze. That is why the answer is a platform and not a patch to TensorBoard.

## 4 What it makes possible

*(Primary audience: everyone — this is the payoff section, and it is deliberately placed before any mechanism. Screenshots/short clips go here in the published version; each example below exists and runs today.)*

The narrow reading of "tensor visualization" is heatmaps. The consequential reading is: **the model's live internal state is in the same address space as a renderer**, so an applet can compute *any* transformation of it — on the GPU, mid-training — and draw the result the same frame. The tensor is the source; the visualization is derived (Figure 2):

- **EmbedScope — geometry of learning.** A digit-classifier with a 3-neuron bottleneck; its activations *are* 3-D coordinates. Two thousand test digits render as a live point cloud that visibly reorganizes from one blob into ten lobes as training runs. Hovering any point shows that digit's image and the model's current guess.

- **GPTScope — a language model, transparent.** A small GPT trains live on Shakespeare. Attention heads render as sharpening heatmaps; sixteen heads reduce to two derived statistics and migrate across a scatter plot as they specialize; a logit lens renders "when does the model decide?" as a colored text grid; generated samples evolve from gibberish to iambic cadence over three minutes.

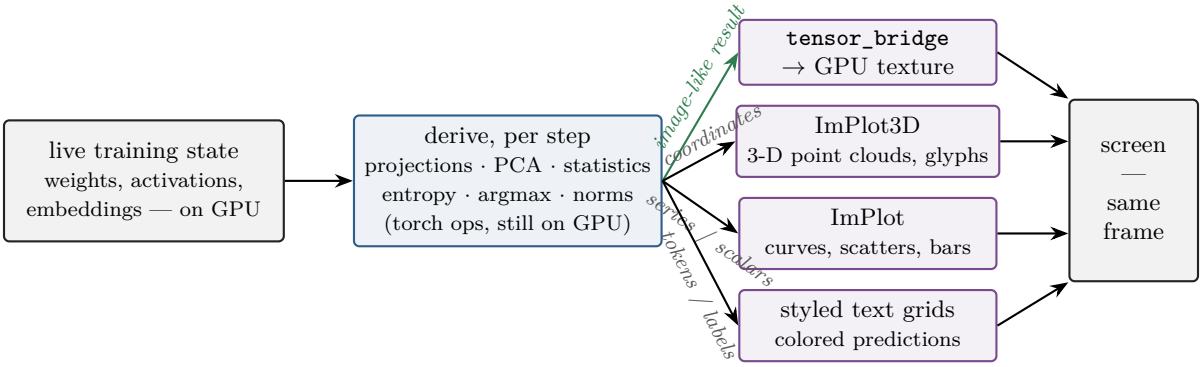


**Figure 2:** The tensor is the source; every visualization is a per-step derivation over live state. Dense image-like results take the zero-copy bridge; coordinate-style outputs flow through plotting libraries as small CPU arrays, and the routes compose freely.

- **Geometry, live and zero-copy — the completed ladder.** A network's *output* can be a solid, not just an image: MeshScope draws a learned 2-D surface — Lambert-lit, loss-colored, wireframe overlay — straight from the GPU memory the optimizer writes, with zero copies of the geometry ever made; paint the target with the mouse and the net visibly chases the edit. On that base the ladder now runs to its top rungs. **TwinScope** drapes a live cotangent-Laplacian heat field (an MLP chasing a simulation) over a CAD heatsink housing at texture resolution — state painted on a twin's shape (`geometry.v1_2`, textures-on-meshes). **instance_scope** renders one mesh times an imported `(N,16)` device pose tensor (optional per-instance tint) as $N$ objects on a 1–5,000 slider in a *single* instanced draw — "1,000 objects, 1 draw call, 0 mesh copies" (`geometry.v1_3`, instanced populations). Every rung — images → points → connected shapes → state-on-shapes → populations — is byte-exact zero-copy on *both* ecosystems (Metal/Apple Silicon and Vulkan/CUDA on an RTX 500 Ada; the v1_2 rungs 2026-07-10, the v1_3 rungs 2026-07-11), behind one frozen additive C ABI.
- **Per-step surfaces.** Convolution kernels and projection matrices update ∼8 times per second — every optimizer step, not every eval — because the cost of showing a tensor no longer includes a trip through the CPU.

Two properties make these more than demos:

- **Identical code, three backends.** The same applet binary renders via zero-copy Metal on a Mac, Vulkan/CUDA interop on a Windows/NVIDIA machine, and a CPU-staged OpenGL fallback anywhere else. The worst case is a working slow path, never a broken one.
- **Honest labeling.** The status line tells you which path ran ("GPU-resident — zero CPU staging" vs "CPU-staged fallback"). The system never claims a fast path it didn't take.

## 5 The business significance

*(Primary audience: business. The argument is cost-of-blindness + deleted infrastructure + portability = market coverage, with verification as the de-risking close.)*

**The cost of training blind.** GPU time is the dominant marginal cost of model development, and the most expensive failure mode is the one discovered late: a run that was doomed at step 500 but ran for twelve hours because nobody could see inside it. Sparse, laggy observability is

not an inconvenience — it is a multiplier on every failed run. Same-frame visualization moves failure detection from *post-mortem* to *while it is happening*, on the developer's own machine.

**Deleted infrastructure.** The incumbent architecture is a logging library + a disk format + a server process + a browser stack. Caliper is one native application; the visualization "pipeline" is a function call. For teams, that is a support surface and a security surface that simply ceases to exist; for laptops-and-workstations development (where an increasing share of iteration happens, especially on Apple Silicon), it is the difference between tooling you run and tooling you *are running a service for*.

**Portability as market coverage.** The two hardware ecosystems that matter — Apple Silicon for local development, NVIDIA for serious training — have *opposite* memory architectures (unified vs. discrete). Caliper's contract never names a graphics API, which is why the NVIDIA implementation landed without changing a single applet, and why the same argument extends to future backends. A competitor must solve both memory models *and* reproduce the frozen-contract discipline to match the portability claim.

**De-risked, not promised.** Every rendering path is verified byte-for-byte against a CPU reference implementation by automated tests on real hardware on both platforms. "It works on both" is a CI gate, not a roadmap item.

*(Deliberately out of scope for v0.1 of this draft: pricing/packaging, cloud story, team-scale collaboration. Flag for a later revision once positioning is decided.)*

# 6 The solution architecture

*(Primary audience: research/engineering. From here down the paper earns the claims made above. Tone shift is intentional and announced by the structure.)*

## 6.1 One contract, three backends

Applets — the visualization programs — never touch a graphics API. They fill a plain C struct (`CaliperTensor`: data pointer, shape, strides, dtype, and a `device` field saying where the bytes live) and hand it to a frozen service ABI, `tensor_bridge.v1`. Behind the host's renderer seam, a backend turns it into a texture however that platform does it best (Figure 3):

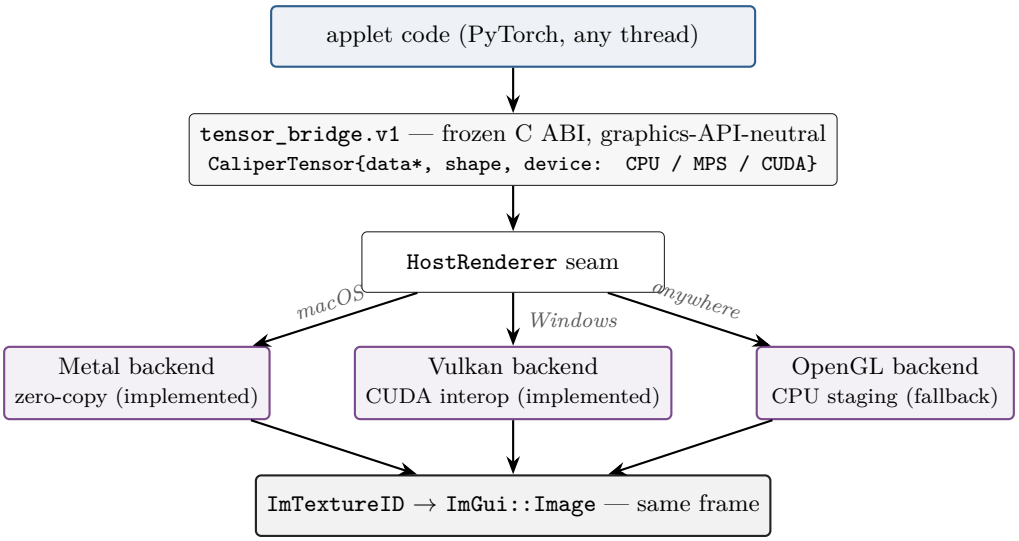


**Figure 3:** One frozen contract; the renderer stays swappable forever. The NVIDIA backend landed without changing a single applet.

| Backend | Mechanism | Data crossings |
|---|---|---|
| Metal / MPS (Apple Silicon) | unified memory: the tensor's storage *is* an `MTLBuffer`; handoff is a pointer cast + safety rules | 0 host copies; 1 on-GPU blit |
| Vulkan / CUDA (Windows/NVIDIA), arbitrary tensor | Vulkan exports a VRAM buffer, CUDA imports it; one in-VRAM `cuMemcpyDtoD`; buffer→image pass | 0 host copies; 1 in-VRAM copy |
| Vulkan / CUDA, `alloc_shared` | the tensor's backing store *is* the interop buffer; kernels write texture memory in place | 0 host copies; **0 in-VRAM copies** |
| Vulkan / CUDA, exportable-pool tensor (bridge v1.2, §6.5) | the applet's pool block is imported into Vulkan once; the pass reads the tensor at its byte offset, in place | 0 host copies; **0 in-VRAM copies** |
| OpenGL / CPU (anywhere) | CPU colormap + one texture upload | 1 host staging + 1 upload |

**Table 1:** The data-crossing ladder, best to worst. Every row is implemented; the native rows are pixel-exact-verified on real hardware.

The returned texture id casts directly to an `ImTextureID`, and the applet draws it with `ImGui::Image` in the same frame.

**Definition discipline.** "Zero-copy" in this paper means: the tensor's *data* never leaves the GPU and never transits host memory. One GPU-side buffer→texture pass remains (samplers read textures, not raw buffers); it runs at memory bandwidth, on-device. For an *arbitrary* framework-allocated CUDA tensor, one in-VRAM copy is the floor (the framework's caching allocator does not produce exportable allocations); the literal zero-copy rungs are `alloc_shared`, where training kernels write the texture's own backing memory, and the exportable pool (§6.5), where the tensor is *born* in memory the renderer can import. The paper — like the product's status lines — reserves each label for the path that earns it.

### 6.2 Apple Silicon: aliasing under unified memory

CPU and GPU share one pool of physical RAM, so a PyTorch MPS tensor's storage already is the object Metal renders from. What makes the cast *safe* rather than merely possible: the frozen struct has no storage-offset channel, so the adapter **rejects** non-contiguous tensors and views with nonzero storage offsets rather than silently copying or — worse — silently addressing the wrong texels. Repairs (`.contiguous()`, `.clone()`) happen in applet code, where their cost is visible. A GPU compute pass applies colormaps for float heatmaps; a blit path handles direct RGBA. Figure 4 shows the handoff.

### 6.3 Windows/NVIDIA: external-memory interop

Discrete GPUs separate VRAM from system RAM, so zero-copy means *keep it in VRAM and make the two APIs share the allocation.* The direction is the non-obvious part: the **renderer exports** (Vulkan external memory, opaque Win32 handle) and **CUDA imports** — because the training framework's allocator can't export. Device pairing is UUID-driven (the Vulkan device and CUDA device must be the same physical silicon; a hybrid-GPU mismatch disables interop rather than corrupting). Colormapping runs as a SPIR-V compute pass byte-identical to the CPU reference. The host loads the CUDA *driver* API at runtime from the system's own driver — no CUDA toolkit dependency; machines without NVIDIA hardware build and run unchanged. Figure 5 shows the flow.

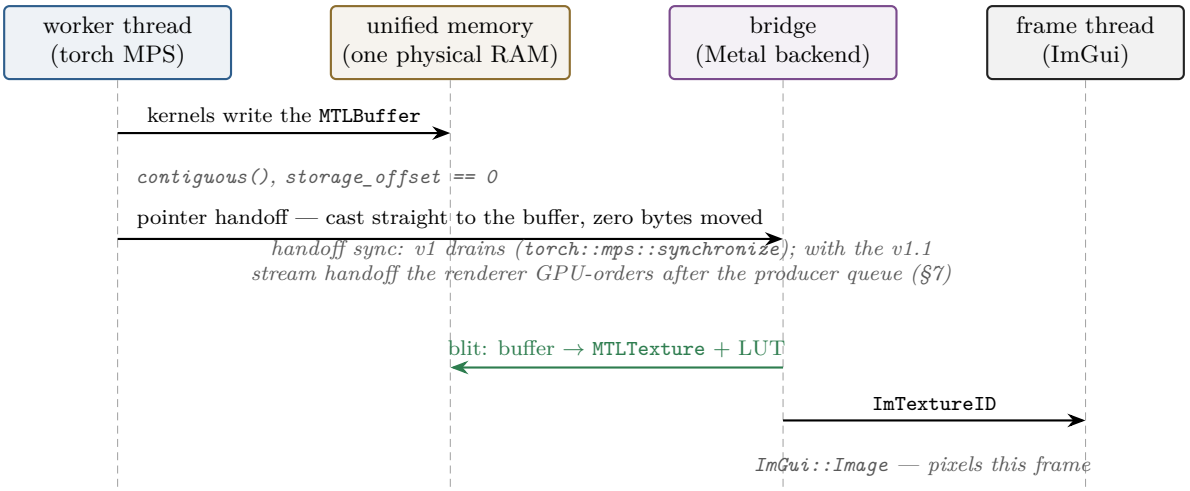


**Figure 4:** The Apple Silicon handoff: the tensor and the texture source are the same physical memory; safety comes from rejection rules and the negotiated sync ladder, not from copies.

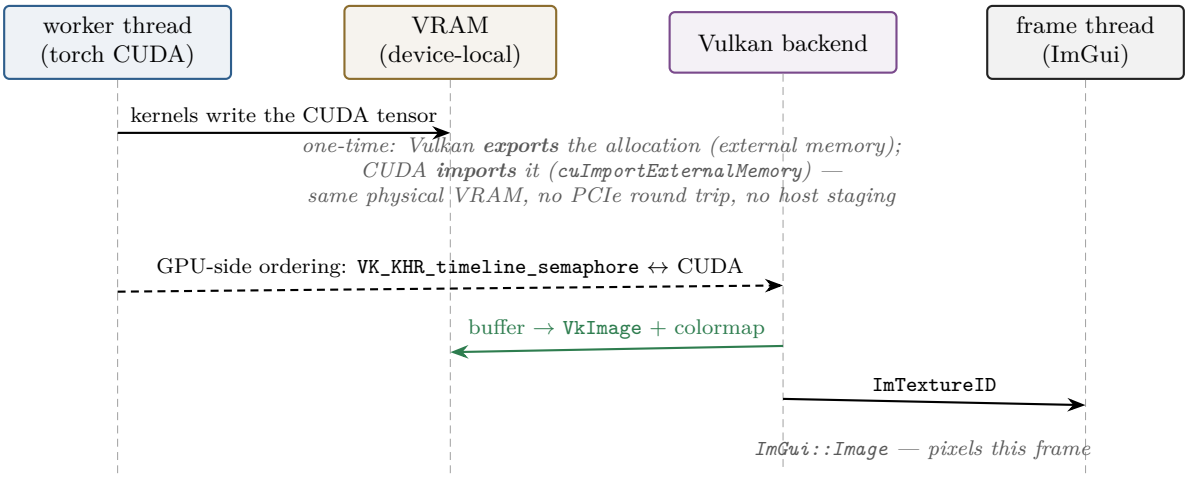


**Figure 5:** The Windows/NVIDIA path: one VRAM allocation shared across APIs, ordered by GPU-side semaphores; no CPU thread waits on the hot path.

### 6.4 The degradation ladder

Every acceptance failure — foreign device, non-contiguous layout, absent interop, exotic allocator — returns `false` and takes the next rung down, ending at the CPU-staged path. A failed fast path is a staged frame, never a crash and never a wrong image. This is why the same applet is demoable on a MacBook, a gaming PC, and a VM.

### 6.5 Removing the arbitrary-tensor floor: the exportable pool (bridge v1.2)

Table 1's "arbitrary tensor" row carries one in-VRAM copy per update because exportability is an *allocation-time* property: memory obtained from the framework's default allocator cannot be retrofitted with a shareable handle. Bridge v1.2 therefore moves the fix to allocation time. The SDK's `ExportablePool` is a torch memory pool whose blocks are created with `cuMemCreate` and a shareable OS handle; an applet materializes exactly the tensors it wants to display inside the pool's scope, and everything else in the process keeps the default allocator. The host imports each block into Vulkan *once* (`import_allocation`) and thereafter updates textures *directly from the block at the tensor's byte offset* (`update_texture_from_alloc`) — the per-update in-VRAM copy no longer exists (Figure 6).

The revision is additive (the v1/v1.1 tables are untouched; hosts advertise a capability bit)

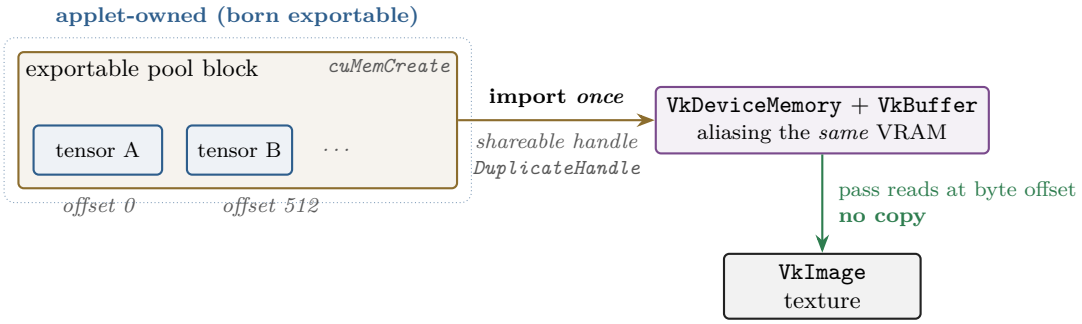


**Figure 6:** Bridge v1.2: tensors born in the exportable pool update textures with zero copies of their data. Blocks are 2 MiB-granularity allocations; torch sub-allocates tensors at 512-byte alignment, which satisfies the renderer's offset-alignment gate.

and fails closed at every gate: a declined import, a misaligned offset, an out-of-bounds window, or a released allocation each return `false` and the applet falls back to the copying path — never a crash, never a wrong image. The floor of Table 1 is thus *per allocation origin*: it persists only for memory born unshareable. The path is hardware-verified: byte-exact readback at multiple offsets through the imported block, and a live training applet whose status line reports *"zero-copy (imported pool)"* only while the imported path is actually the one on screen. (A companion technical report, `docs/report/zerocopy-import-v12`, documents the design and its verification in full.)

# 7 Synchronization: the part that bites

*(Primary audience: research/engineering. This section carries the paper's credibility with systems readers — it is where most naive implementations die, and we present it races-first.)*

Sharing memory between a producer that is still writing and a consumer that renders is only correct if *when* is as disciplined as *where*. Caliper's answer is a two-rung negotiated ladder:

- **v1 — drain.** Synchronize the producer device fully before handoff (sync-then-update). Always correct; costs one full-device barrier, paid once at the handoff, never per frame.
- **v1.1 — stream-ordered handoff.** The adapter publishes the producer's queue/stream in the tensor struct; the renderer GPU-orders its work after the producer's (per-texture `MTLSharedEvent` on Metal; a shared timeline semaphore riding the producer's CUDA stream on Vulkan). No CPU thread waits on the hot path. Hosts advertise the capability; adapters degrade to the drain automatically.

Two hard-won findings, offered as cautionary results:

1. **PyTorch's public MPS stream calls are not internally serialized** (we established this by disassembly after crashes in three different costumes). Handing off while the training thread encodes corrupts command-buffer state. All MPS-touching handoff work must run as one block on the framework's own stream dispatch queue.
2. **CUDA's thread-safety is contractual — we pinned it anyway.** The MPS lesson was precisely that "should be safe" is not evidence; a 500-handoff concurrent-training stress test holds the property empirically.

And one honest measurement: eliding the drain did **not** improve training throughput ($\approx 0$ delta on the benchmark machine — the training loop's own per-step `loss.item()` sync

dominates). The verified win is ordering correctness plus removal of frame-thread stalls. Papers in this genre routinely imply throughput wins they haven't measured; we measured, and report what we found.

## 8 Verification: pixel-exact or it didn't happen

*(All audiences, one page. For business readers this is the de-risking section; for researchers it is the methods section.)*

The claim "the GPU path is equivalent to the reference" is a *byte* claim, tested as one. A single CPU function is the source of truth for colormapping; every GPU implementation (Metal compute, Metal blit, SPIR-V compute, Vulkan copy) must read back **byte-identical** output across adversarial sizes (non-multiple-of-16, ragged shapes), NaN inputs, and degenerate ranges — in an automated windowed test harness, per backend, every CI run, including on real NVIDIA hardware. The stream-ordered path adds a burst test: eight overlapping updates ordered only by semaphores must land byte-exact. The imported-allocation path (§6.5) adds rows that must read back byte-exact *at nonzero byte offsets* through an imported block. Reading device buffers back for these comparisons is itself platform-shaped: torch MPS tensors import as *private-storage* Metal buffers whose bytes the host cannot read directly, so the gate stages them through a blit — the same shape the discrete-VRAM Vulkan path already required. Failure modes are designed to be loud: acceptance violations log and reject rather than render a plausible-but-wrong image.

## 9 Limits and non-claims

*(Research audience explicitly; business audience implicitly — a paper that states its limits is a paper whose claims can be trusted.)*

- Arbitrary framework-allocated CUDA tensors carry a one in-VRAM-copy floor **unless allocated from Caliper's exportable pool, which removes it**; the floor persists only for memory born unshareable.
- The v1 contract requires contiguous, offset-0 tensors; views are rejected, not repaired.
- Dense outputs (heatmaps, feature maps, attention grids) and the full geometry ladder (point clouds, lines, indexed meshes, textures-on-meshes, instanced populations — `geometry.v1` through `v1_3`, R0–R3) all ride zero-copy paths, byte-exact on both ecosystems; only small derived series (curves, scatters, glyph clouds) still flow through plotting libraries as CPU arrays — kilobytes, and the routes compose freely.
- The OpenGL fallback stages through the CPU by design; it exists so the worst case is slow, not broken.
- Same-frame visualization does not by itself accelerate training; its value is observational (§7's measurement note).
- Single-machine, in-process by design. Remote/cluster training is a different problem (and largely re-introduces the round trip that distributed tooling rightly accepts); this paper claims the local loop.

## 10 Implications and outlook

For the **general reader**: instruments change what gets studied. When the microscope got a video feed, biology got cell dynamics as a field.

For **business**: the local development loop is where iteration speed compounds; tooling that deletes seconds from every look-inside changes how often people look.

For **research**: most interpretability is post-hoc — artifacts of finished training. Per-step, in-the-loop observation makes *training dynamics* (representation reorganization, head specialization timing, loss-spike anatomy) a first-class observable. The platform's applet model means a new visualization is a new derivation over live state — a torch op and a draw call — not a new pipeline.

On the platform's own trajectory: the core that does all of this — the applet contract, the host-neutral services, and the zero-copy renderer with its completed geometry ladder — is now extractable as an embeddable library (`libcaliper`) behind a small C ABI, so a second host can vend the same applets without owning the machinery. A GLFW-free native host already runs the applets zero-copy on Apple Silicon (Metal/MPS); the Windows/NVIDIA embedding path is written but its hardware verification is pending the next Windows session, and a full sibling host remains a design-gated future, not a claim.

# A The bridge contract (abridged)

The load-bearing property is what the contract does *not* contain: no graphics-API type appears anywhere. Textures cross the ABI as an opaque 64-bit id; the host keeps an id → backend-handle table, so the renderer behind the seam is swappable without touching an applet. The struct below is frozen (immutable once published); later revisions (v1.1's capability query and stream field, v1.2's imported allocations, §6.5) are additive — prefix-identical structs and new capability bits, so old applets run on new hosts and vice versa.

```
typedef uint64_t CaliperTextureId;   /* opaque; 0 = invalid; castable to
                                        ImTextureID for ImGui::Image */

typedef struct CaliperTensorBridgeV1 {
    uint32_t struct_size;

    /* Mirror a 3-D (H,W,C<=4) u8 tensor as a texture. GPU-resident on the
       native backends; CPU-staged on the GL fallback. 0 on failure. */
    CaliperTextureId (*texture_from_tensor)(const CaliperTensor* t,
                                            uint32_t flags);

    /* Re-upload into an existing texture (same shape/dtype). */
    bool (*update_texture)(CaliperTextureId tex, const CaliperTensor* t);
    void (*release_texture)(CaliperTextureId tex);

    /* Colormap a 2-D (H,W) f32 tensor through a built-in LUT, scaling
       [vmin,vmax] -> [0,1]. Identical numeric output on every backend. */
    CaliperTextureId (*texture_from_tensor_mapped)(const CaliperTensor* t,
                                                   int32_t colormap,
                                                   float vmin, float vmax,
                                                   uint32_t flags);

    /* Literal zero-copy: allocate tensor memory that IS the texture's
       backing store; the applet's kernels write it in place. */
    bool (*alloc_shared)(CaliperDType dtype, int32_t ndim,
                         const int64_t* shape,
                         CaliperTensor* out_tensor,
                         CaliperTextureId* out_texture);
    void (*free_shared)(CaliperTextureId tex);
} CaliperTensorBridgeV1;
```

Acceptance rules (violations return 0/`false` and log — never a misinterpreted texture): 2-D f32 for the mapped path; 3-D u8 for the direct path; contiguous row-major layout; the tensor's

device must be the CPU or the active backend's device. The v1.2 additions extend the same table with the imported-block path of §6.5,

```
import_allocation   release_allocation   update_texture_from_alloc
```

behind the `CALIPER_BRIDGE_CAP_IMPORT_ALLOC` capability bit.

# B Verification matrix

Every row below shipped and is green as of July 2026. "Byte-exact" means `==` on the RGBA8 readback against the single CPU reference function, no tolerances.

| Path | Test | Hardware | Status |
|---|---|---|---|
| Metal compute (f32 colormap) | §16 matrix: 3 colormaps, ragged sizes, NaN, degenerate ranges; byte-exact | Apple M-series | green |
| Metal blit (u8 RGBA) | direct-upload matrix; byte-exact | Apple M-series | green |
| Metal stream-ordered handoff | per-texture `MTLSharedEvent` ordering; concurrent-training stress | Apple M-series | green |
| Vulkan CPU-staged (portable) | the same §16 matrix on the Vulkan backend | any ICD (CI: lavapipe) | green |
| Vulkan/CUDA interop (arbitrary tensor) | device tensor → `"compute"` path; byte-exact at 4×4, 5×3, 17×9 | RTX 500 Ada | green |
| Vulkan/CUDA `alloc_shared` | in-place device write reads back pixel-exact; no copy | RTX 500 Ada | green |
| Vulkan/CUDA pipelined sync | 8-update burst ordered only by timeline semaphores; byte-exact final frame | RTX 500 Ada | green |
| Imported allocation, f32 (§6.5) | byte-exact at offsets 0 and 512 through `"compute-imported"` | RTX 500 Ada | green |
| Imported allocation, u8 | `"blit-imported"` at offset 512; byte-exact | RTX 500 Ada | green |
| Imported allocation, negative rows | misaligned offset, out-of-bounds window, released id: `false`, pixels untouched | RTX 500 Ada | green |
| Imported allocation, lifecycle | 50× import/release, ids strictly increasing; live-app cancel/relaunch | RTX 500 Ada | green |
| OpenGL fallback | the §16 matrix, CPU-staged; live-app fallback run | any | green |

**Table 2:** Path × test × hardware, distilled from the internal test documentation (unit + gfx + torch suites; the gfx suite alone is 22 cases / 1030 assertions as of this draft).

# C Glossary for the general reader

**Tensor** a multi-dimensional array of numbers — the universal container for everything inside a neural network.

**GPU / VRAM** the processor that draws your screen and (in ML) does the training math; VRAM is its private, very fast memory.

**Unified memory** Apple Silicon's design where CPU and GPU share one pool of physical RAM, so "moving" data between them can be free.

**Texture** the GPU-native image format the renderer samples from when putting pixels on screen.

**Frame** one full refresh of the screen (typically 1/60th of a second); "same frame" means computed and displayed with no refresh in between.

**Interop** two APIs agreeing to reference the same memory instead of copying between their private worlds.

**Semaphore** a GPU-side signal used to order work ("draw only after the training kernel finished writing") without making a CPU thread wait.